Team Winners

OOP Team Project | 2023-1학기

2021043981 나영하
2022065541 백상준
2022093854 허정후

목차

01 목표 설정

02 전략

03 구현 내용

04 추가 구성
GUI

Team Winners

목표 설정

상대 팀의 수를 어떻게 읽어낼 것인가?

상대에게 어떻게 우리팀의 전략을 읽히지 않을 수 있을까?

읽어낸 상대 팀의 수를 어떻게 활용할 것인가?

TeamWinners

전략

1

손실 최소화 전략

2

이익 최대화 전략

3

상대 패턴 분석

4

무작위 전략

TeamWinners

전략

1

손실 최소화 전략

2

이익 최대화 전략

	player2 가위	player2 바위	player2 보
player1 가위	0.5	-0.2	0.7
player1 바위	0.3	0.8	-0.1
player1 보	0.2	0.4	0.4

TeamWinners

전략

1

손실 최소화 전략

2

이익 최대화 전략

	player2 가위	player2 바위	player2 보
player1 가위	0.5	-0.2	0.7
player1 바위	0.3	0.8	-0.1
player1 보	0.2	0.4	0.4

TeamWinners

전략

1

손실 최소화 전략

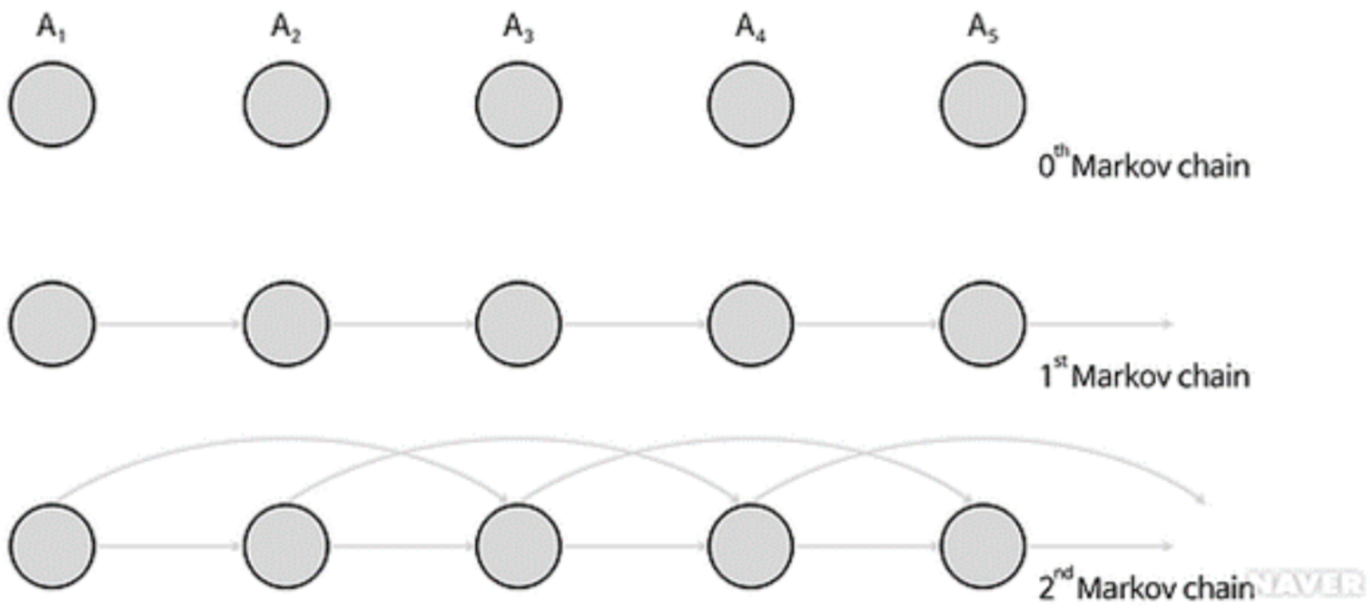
2

이익 최대화 전략

	player2 가위	player2 바위	player2 보
player1 가위	0.5	-0.2	0.7
player1 바위	0.3	0.8	-0.1
player1 보	0.2	0.4	0.4

TeamWinners

전략



3

상대 패턴 분석

TeamWinners

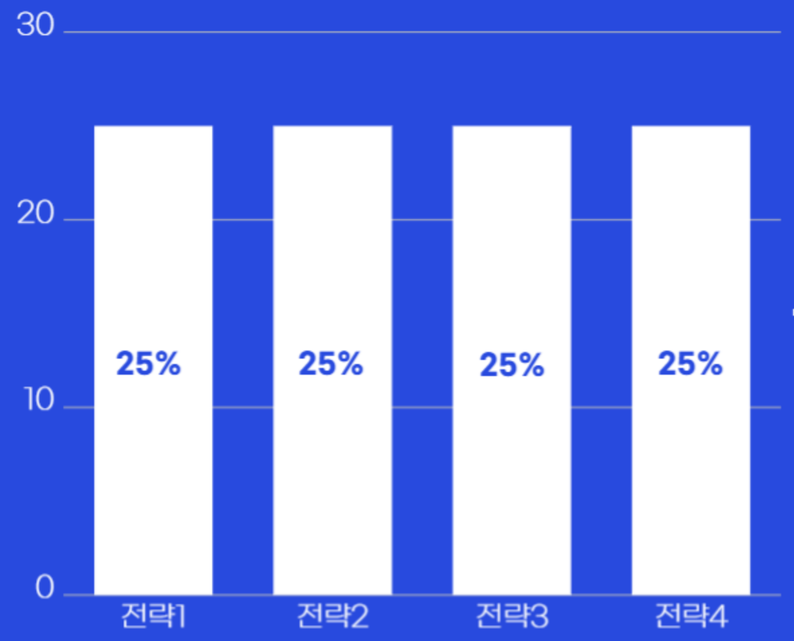
전략

4

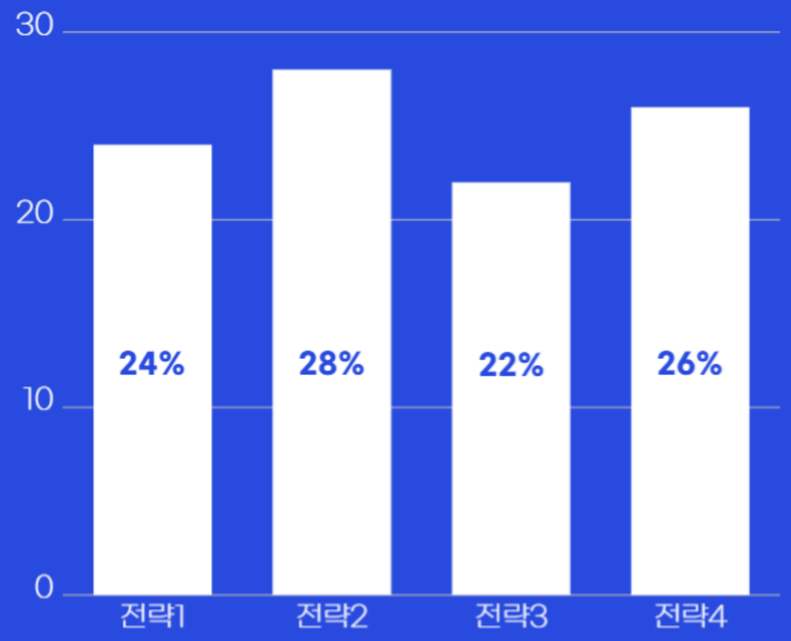
무작위 전략

TeamWinners

전략 비율



전략 별 비율 초기 설정



전략 별 비율 조정

조정 구간 : 500회
최대 조정 정도 : 0.3

TeamWinners

구현 내용

전략1

```
public int MinimumDamage(float[][] gainMatrix, int playermode) { // 피해를 본다면 가장 적게 보거나, 안정적으로 점수를 획득할 수 있도록 하는 전략
    float Column_Element1 = 0, Column_Element2 = 0, Column_Element3 = 0;
    float Row_Element1 = 0, Row_Element2 = 0, Row_Element3 = 0;
    int FirstUserMove, SecondUserMove;
    int FinalNumber = 0;


    if (playermode == 0) { // RowSide기준
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) { //i번째 열의 값을 모두 저장해준다.
                if (j == 0) {
                    Column_Element1 = gainMatrix[i][j];
                } else if (j == 1) {
                    Column_Element2 = gainMatrix[i][j];
                } else if (j == 2) {
                    Column_Element3 = gainMatrix[i][j];
                }
            }
            // 첫번째, 곱이 음수인경우 -> 두개가 양수이고, 하나가 음수인 경우 두번째, 전부 양수인 경우
            if (((Column_Element3 * Column_Element2 * Column_Element1 < 0) && !(Column_Element1 < 0 && Column_Element2 < 0 && Column_Element3 < 0)) || ((Column_Element1 > 0)
                if (Column_Element3 * Column_Element2 * Column_Element1 < 0) { // 첫번째 경우
                    FirstUserMove = i; // 해당하는 열을 저장
                    FinalNumber = FirstUserMove;
                } else if (((Column_Element1 > 0) && (Column_Element2 > 0) && (Column_Element3 > 0))) { // 두번째 경우
                    SecondUserMove = i; // 해당하는 열을 저장
                    FinalNumber = SecondUserMove;
                    return FinalNumber; // 이 경우에는 반드시 이 값이 실행되어야 하므로 return을 넣어준다.
                }
            }
        }

    } else if (playermode == 1) { // ColumnSide기준
        for (int j = 0; j < 3; j++) {
            for (int i = 0; j < 3; j++) { // i번째 행의 값을 모두 저장해준다.
                if (i == 0) {
                    Row_Element1 = gainMatrix[i][j];
                } else if (i == 1) {
                    Row_Element2 = gainMatrix[i][j];
                } else if (i == 2) {
                    Row_Element3 = gainMatrix[i][j];
                }
            }
            // 첫번째 전부 음수인 경우(ROW이므로) 두번째, 곱이 양수인 경우-> 두개가 음수이고, 하나가 양수인 경우
            if ((Row_Element1 < 0 && Row_Element2 < 0 && Row_Element3 < 0) || ((Column_Element3 * Column_Element2 * Column_Element1 > 0) && !(Column_Element1 > 0 && Column_
                if (Row_Element1 < 0 && Row_Element2 < 0 && Row_Element3 < 0) { // 첫번째 경우
                    FirstUserMove = j; // 해당하는 행을 저장
                    FinalNumber = FirstUserMove;
                    return FinalNumber;  // 이 경우에는 반드시 이 값이 실행되어야 하므로 return을 넣어준다.
```

```
                } else if ((Column_Element3 * Column_Element2 * Column_Element1 > 0) && !(Column_Element1 > 0 && Column_Element2 > 0 && Column_Element3 > 0)) { // 두번째 경
                    SecondUserMove = j; // 해당하는 행을 저장
                    FinalNumber = SecondUserMove;
                }
            }
        }
    }

    return FinalNumber;
```

TeamWinners

구현 내용

전략2

```java
public int maxBenefit(int playermode) { //이득을 최대로 하자 = 절댓값이 큰 성분을 찾게 됨
    float tempMax = 0;  //tempMax의 초기값을 0으로 설정
    int maxIndexRow = 0;  //내가 row-side에 있을 때 내가 내게 될 move의 index
    int maxIndexColumn = 0;   //내가 column-side에 있을 때 내가 내게 될 move의 index
    if (playermode == 0) {  //내가 row-side일 때는 절댓값이 가장 큰 양수 성분을 찾게 됨(가장 큰 수)
        for (int i = 0; i < MatrixForClass.length; i++) {
            for (int j = 0; j < MatrixForClass.length; j++) { //행렬의 원소를 모두 비교해서 가장 큰 점수 찾음(양수)
                if (tempMax < MatrixForClass[i][j]) {        //가장 큰 점수를 찾기 때문에 처음의 기준을 0으로 설정해도 됨
                    tempMax = MatrixForClass[i][j];          //더 큰 원소 있으면 tempMax 계속 업데이트
                    maxIndexRow = i;                 //더 작은 원소 나올 때마다 그 원소가 포함된 row의 index 저장
                }
            }
        }
        return maxIndexRow; //가장 큰 성분이 포함된 row의 index 반환
    } else {  //내가 column-side일 때는 절댓값이 가장 큰 음수 성분을 찾게 됨(가장 작은 수)
        for (int i = 0; i < MatrixForClass.length; i++) {     //column-side에서는 모든 성분의 부호가 반대
            for (int j = 0; j < MatrixForClass.length; j++) {   //행렬의 원소를 모두 비교해서 가장 작은 점수 찾음(음수)
                if (tempMax > MatrixForClass[i][j]) {        //가장 작은 점수를 찾기 때문에 처음의 기준을 0으로 설정해도 됨
                    tempMax = MatrixForClass[i][j];       //작은 원소 있으면 tempMax 계속 업데이트
                    maxIndexColumn = j;         //작은 원소 나올 때마다 그 원소가 포함된 column의 index 저장
                }
            }
        }
        return maxIndexColumn;  //가장 작은 성분이 포함된 column의 index 반환
```

TeamWinners

구현 내용

전략3

```java
public int Markov(int playermode, int oppmove, int oppmove1, int oppmove2) {
    int decision = 0;

    for (int i = 0; i < index - 2; i++) {
        if (opp[i] == oppmove) {
            if (opp[i + 1] == oppmove1 && opp[i + 2] == oppmove2 && opp[i + 3] == ROCK) {
                RockCount++;
            } else if (opp[i + 1] == oppmove1 && opp[i + 2] == oppmove2 && opp[i + 3] == Scissors) {
                ScissorCount++;
            } else if (opp[i + 1] == oppmove1 && opp[i + 2] == oppmove2 && opp[i + 3] == Paper) {
                PaperCount++;
            }
        }
    }
```

```java
//상대방꺼 분석해서 내가 낼 거 결정
if (playermode == 1) { //내가 col 모드
    if (RockCount < PaperCount) {
        if (PaperCount < ScissorCount) { //상대방이 가위 낼 확률이 가장 높을 때
            decision=row1min_index ;
        } else { //바위낼 확률이 가장 높을 때
            decision = row2min_index;
        }
    } else {
        if (RockCount > ScissorCount) { //
            decision = row0min_index;
        } else {
            decision = row1min_index;
        }
```

```java
else if(playermode==0){ //내가 row모드
    if (RockCount < PaperCount) {
        if (PaperCount < ScissorCount) { //상대가 가위 낼 확률이 젤 높을 때 (가위보바위
            decision=column1max_index ; //2열에서 젤 큰 값 가져가야함
        } else { //보 낼 확률 젤 높을 때 (보 가위 바위 or 보 바위 가위
            decision = column2max_index; //3열에서 젤 큰 값으로 내가 낼 거 결정
        }
    } else { //바위가 보 보다 확률 크고
        if (RockCount > ScissorCount) { //바위가 확률 제일 클 때 (바위 가위 보 or 바위 보 가위
            decision = column0max_index;
        } else { //(가위 바위 보
            decision = column1max_index; //가위 열. 2열에서 가장 큰값
        }
    }
}
return decision;
```

TeamWinners

구현 내용

전략4

```
public int RandomNumberForRSP(int playermode) {
    int high = 2;
    int low = 0;
    int myRandnum = (int) (Math.random() * (high - low + 1));
    return myRandnum;
}
```

TeamWinners

구현 내용

확률 조정

```
public void MovementOfPercentage() {
    float RandomNumberForMovement = (float) (Math.random() * 0.3); //0~0.3까지의 랜덤한 값 저장
    if (AverageOfStrategy1 > AverageOfStrategy2 && AverageOfStrategy1 > AverageOfStrategy3 && AverageOfStrategy1 > AverageOfStrategy4) { //1번 전략이 가장 우수
        if (AverageOfStrategy4 < AverageOfStrategy2 && AverageOfStrategy4 < AverageOfStrategy3) {//4번 전략이 가장 실적이 좋지 못할때
            FirstPercentage += RandomNumberForMovement;//위에서 생성한 랜덤한 조정 값을 플러스하여 1번 전략의 실행 확률을 높임
            SecondPercentage += RandomNumberForMovement;//나머지 값들은 일정하게 비율을 가질 수 있도록 동일하게 높여줌
            ThirdPercentage += RandomNumberForMovement;//그럼 마지막 전략4번의 비율은 자동적으로 올려준만큼 감소됨
        }//1번이 best &3번이 worst
        else if (AverageOfStrategy3 < AverageOfStrategy2 && AverageOfStrategy3 < AverageOfStrategy4) {
            FirstPercentage += RandomNumberForMovement;
            SecondPercentage += RandomNumberForMovement;
        }//1번이 best&2번이 worst
        else if (AverageOfStrategy2 < AverageOfStrategy3 && AverageOfStrategy2 < AverageOfStrategy4) {
            FirstPercentage += RandomNumberForMovement;
        }
    }//2번이 best
    else if (AverageOfStrategy2 > AverageOfStrategy1 && AverageOfStrategy2 > AverageOfStrategy3 && AverageOfStrategy2 > AverageOfStrategy4) {
        if (AverageOfStrategy4 < AverageOfStrategy1 && AverageOfStrategy4 < AverageOfStrategy3) {//2번이 best일 때 4번이 worst
            SecondPercentage += RandomNumberForMovement;
            ThirdPercentage += RandomNumberForMovement;
        }
        else if (AverageOfStrategy3 < AverageOfStrategy4 && AverageOfStrategy3 < AverageOfStrategy1) {
            SecondPercentage += RandomNumberForMovement;
        }
        else if (AverageOfStrategy1 < AverageOfStrategy3 && AverageOfStrategy1 < AverageOfStrategy4) {
            FirstPercentage -= RandomNumberForMovement;
        }
    }
    else if (AverageOfStrategy3 > AverageOfStrategy1 && AverageOfStrategy3 > AverageOfStrategy2 && AverageOfStrategy3 > AverageOfStrategy4) {
        if (AverageOfStrategy4 < AverageOfStrategy1 && AverageOfStrategy4 < AverageOfStrategy2) {
            ThirdPercentage += RandomNumberForMovement;
        }
        else if (AverageOfStrategy2 < AverageOfStrategy4 && AverageOfStrategy2 < AverageOfStrategy1) {
            SecondPercentage -= RandomNumberForMovement;
        }
        else if (AverageOfStrategy1 < AverageOfStrategy4 && AverageOfStrategy1 < AverageOfStrategy2) {
            FirstPercentage -= RandomNumberForMovement;
            SecondPercentage -= RandomNumberForMovement;
        }

    }
```

TeamWinners

구현 내용

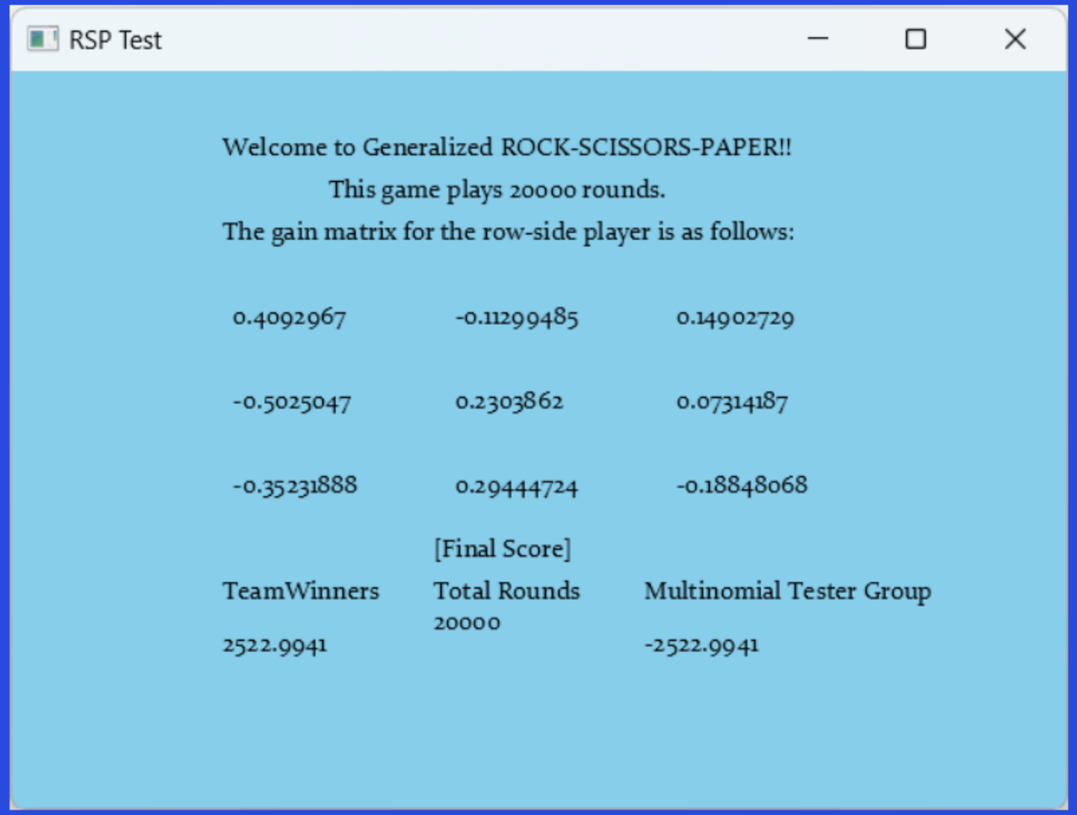
전략 선택

```
if (index >= 100) { //전략 선택을 위해서 랜덤한 숫자가 선택될 수 있도록 위에서 정한
    if (RandomNumberForStrategy < FirstPercentage) { //랜덤 전략이 전략1번의 구간 내에 들어오면
        int NextUserMove = MinimumDamage(gainMatrix: MatrixForClass,playermode);
        CountOfStrategy1++; //전략1번 시행총횟수 증가(=이익최대화전략)
        SumOfStrategy1 += ScoreOfThistime; //이번턴의 점수를 전략1번의 점수에 더해줌(전략1번의 총점 저장)
        AverageOfStrategy1 = SumOfStrategy1/CountOfStrategy1;//전략1번의 평균 저장
    }//첫번째 퍼센트가 랜덤 전략보다 작거나 작고, 랜점 전략은 두번째 퍼센트보다 작으면
    else if (FirstPercentage <= RandomNumberForStrategy && RandomNumberForStrategy < SecondPercentage) {
        int NextUserMove = maxBenefit(playermode);//다음 전략은 최대화전략
        CountOfStrategy2++; //전략2번 총횟수 증가
        SumOfStrategy2 += ScoreOfThistime; //전략2의 총점 저장
        AverageOfStrategy2 = SumOfStrategy2/CountOfStrategy2; //전략2의 평균 저장
    }
    else if (SecondPercentage <= RandomNumberForStrategy && RandomNumberForStrategy< ThirdPercentage) {
        int NextUserMove = Markov(playermode, opp[index - 3], opp[index - 2], opp[index - 1]);
        CountOfStrategy3++;
        SumOfStrategy3 += ScoreOfThistime;
        AverageOfStrategy3 = SumOfStrategy3/CountOfStrategy3;
    }
    else if (ThirdPercentage < RandomNumberForStrateg
        int NextUserMove = RandomNumberForRSP(playerm

        CountOfStrategy4++;
        SumOfStrategy4 += ScoreOfThistime;
```

```
else { //초기에는 마코프 체인이 실행되면 안되기 때문에 index가 100보다 작은 경우에 전략3번은 제외한 내용 구현
    if (RandomNumberForStrategy < 0.33f) { //3개의 전략을 처음에 공평한 내용으로 가져갈 수 있도록
        int NextUserMove = MinimumDamage(gainMatrix: MatrixForClass,playermode); //손실최소화 전략 선택

        CountOfStrategy1++; //1번 증가
        SumOfStrategy1 += ScoreOfThistime;
        AverageOfStrategy1 = SumOfStrategy1/CountOfStrategy1;
    }
    else if (0.33 <= RandomNumberForStrategy && RandomNumberForStrategy < 0.67) {
        int NextUserMove = maxBenefit(playermode);

        CountOfStrategy2++;
        SumOfStrategy2 += ScoreOfThistime;
        AverageOfStrategy2 = SumOfStrategy2/CountOfStrategy2;
    }
    else if (0.67 <= RandomNumberForStrategy && RandomNumberForStrategy < 1.0) {
        int NextUserMove = RandomNumberForRSP(playermode);

        CountOfStrategy4++;
        SumOfStrategy4 += ScoreOfThistime;
        AverageOfStrategy4 = SumOfStrategy4/CountOfStrategy4;
    }
```

TeamWinners

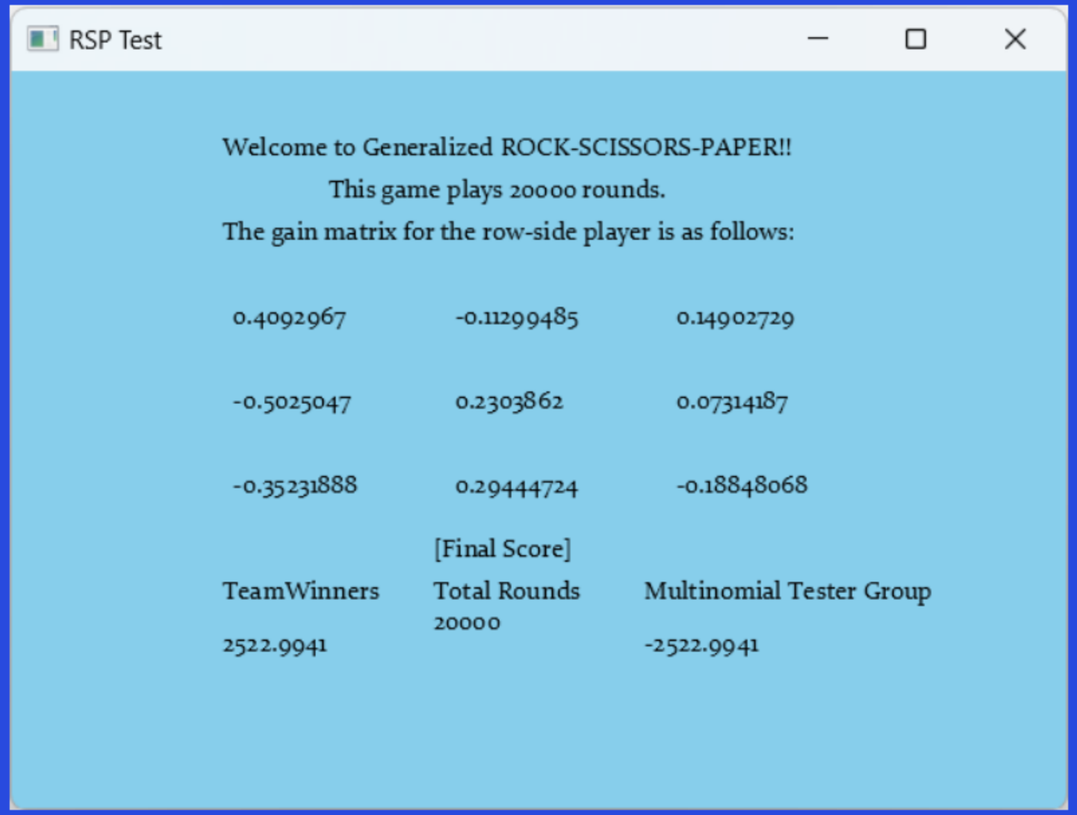
추가 구성 : GUI




```
public void start(Stage primaryStage) {
    //-------------------------------------------
    // Welcome ... as follows : 와 gainmatrix의 원소를 저장한 Text instance 생성.
    // generating Text instances which store the descriptions : "Welcome ~ as follows:", "Total Rounds₩n +
HowManyRuns", "[Final Score]", and each element of gainmatrix
    Text welcome = new Text(100, 40, "Welcome to Generalized ROCK-SCISSORS-PAPER!!"); // coordinates(좌표) :
(100,40)
    Text rounds = new Text(150, 60, "This game plays " + HowManyRuns + " rounds."); // coordinates(좌표) : (150,60)
    Text matrixDescription = new Text(100, 80, "The gain matrix for the row-side player is as follows:"); // coordinates(좌
표) : (100,80)
    Text roundsinfo = new Text(200, 275, "Total Rounds₩n" + HowManyRuns); //coordinates(좌표) : (200,275)
    Text scoreinfo = new Text(200, 250, "[Final Score]"); //coordiantes(좌표) : (200,250)
    //placing each of the elements on the proper location corresponding to the matrix
    //gainmatrix에 대응하는 적절한 위치에 각 원소 위치시키기
    Text[][] elements = new Text[gainmatrix.length][gainmatrix[0].length];
    for (int i = 0; i < gainmatrix.length; i++) {
      for (int j = 0; j < gainmatrix[0].length; j++) {
        elements[i][j] = new Text(105 * (j + 1), 120 + 40 * i, Float.toString(gainmatrix[i][j]));
      }
    }
    //-------------------------------------------
    //------------------------------
    //generating Group instance collecting all the Text instances made in advance
    //이전에 만든 모든 Text instance 모으는 Group instance 생성
    Group root = new Group(welcome, rounds, matrixDescription, elements[0][0], elements[0][1],
        elements[0][2], elements[1][0], elements[1][1], elements[1][2], elements[2][0], elements[2][1], elements[2][2],
        player1name, player2name, roundsinfo, scoreinfo, player1score, player2score);
    //------------------------------
    //generating Scene instance whose width, height, and color are 500, 350, skyblue respectively with root(generated
above) as the paramter for the contents
    //내용물을 위한 paramter로서 위에서 만들어진 root을 가지고 있는 너비 500, 높이 350, 하늘색인 Scene 인스턴스 생성
    Scene scene = new Scene(root, 500, 350, Color.SKYBLUE);
    primaryStage.setTitle("RSP Test"); //setting the title "RSP Test" on the window
                                       //창 이름은 RSP Test
    primaryStage.setScene(scene); //filling the window with the contents from the Scene instance made above
                                  //위에서 만든 scene 인스턴스로 창 채우기
    primaryStage.show(); //showing the GUI for the result of RSP test
                         //GUI 보여주기
  }
```

TeamWinners

추가 구성 : GUI




```
public static void main(String[] args) {
.
.
.
//generating Text instances which store player1name, player2name, player1score, player2score respectively
    //player1name, player2name, player1score, player2score를 각각 저장한 Text 인스턴스 생성
    player1name = new Text(100, 250, rspPlayer1.getYourGroupName());  //coordinates(좌표) : (100,250)
    player2name = new Text(300, 250, rspPlayer2.getYourGroupName());  //coordinates(좌표) : (300,250)
    player1score = new Text(100, 275, Float.toString(rspPlayer1Gain)); //coordinates(좌표) : (100,275)
    player2score = new Text(300, 275, Float.toString(rspPlayer2Gain)); //coordinates(좌표) : (300,275)
launch(args); //start() 메소드 실행. execute start() method
  }
```

감사합니다

OOP Team Project | TeamWinners

2021043981 나영하
2022065541 백상준
2022093854 허정후